



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ Информатика и системы управления (ИУ)

КАФЕДРА _____ Теоретическая информатика и компьютерные технологии (ИУ9)

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3 **по курсу «Алгоритмы компьютерной графики»**

Выполнила: *Зайковская*
Екатерина Владимировна,
ИУ9-42Б

Москва 2024

Задача

Главной задачей лабораторной работы является создание изображения фигуры с возможностью изменения ракурса камеры. Основные принципы реализации сохраняются из лабораторной работы №2 (предыдущей). Поэтому подзадачи следующие:

1. Определить в качестве модели объекта сцены фигуру в соответствии с вариантом.
2. Реализовать изменение ориентации и размеров объекта (навигацию камеры) с помощью модельно-видовых преобразований (управление производится интерактивно с помощью клавиатуры и/или мыши).
3. Предусмотреть возможность переключения между каркасным и твердотельным отображением модели.

Индивидуальный вариант заключается в изображении *эллипсоида*.

Основная теория

Одним из способов задания поверхности является параметрический. При этом координаты точек поверхности рассматриваются как некие функции от двух параметров, пробегающих некоторый набор значений.

Таким способом можно задать некоторый набор точек на задаваемой поверхности, соседние точки при этом отсоят друг от друга на значение шага одного и/или двух параметров, задающих поверхность. Таким образом соседние вершины образуют грань, и поверхность приближается многоугольником с конечным числом вершин, лежащих на заданной параметрически поверхности.

Такое представление удобно для поверхностей второго порядка.

Задание эллипсоида

Эллипсоид - поверхность в трёхмерном пространстве, полученная деформацией сферы.

В декартовых координатах (в системе координат с тремя взаимно перпендикулярными осями x , y , z) эллипсоид задается уравнением:

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} + \frac{z^2}{c^2} = 1$$

Каноническое уравнение эллипсоида

И эллипсоид имеет следующий вид:

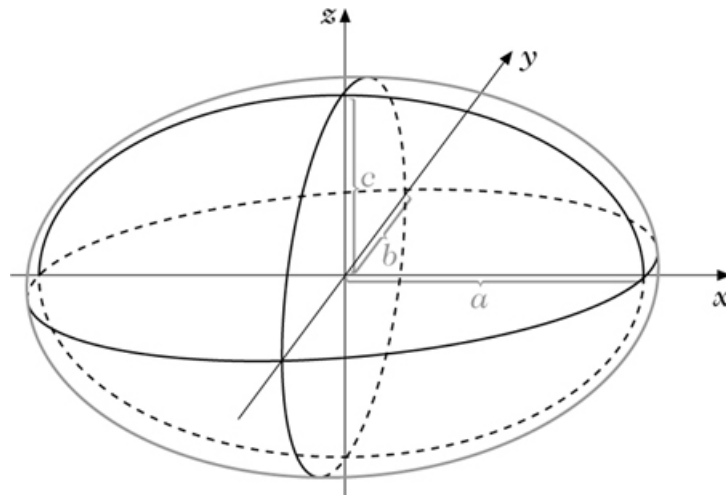


Рис.: Эллипсоид

Параметрически эллипсоид задается следующим образом:

$$\begin{aligned} x &= a \sin(\theta) \cos(\varphi), \\ y &= b \sin(\theta) \sin(\varphi), \\ z &= c \cos(\theta), \end{aligned}$$

$$0 \leq \theta \leq \pi, \quad 0 \leq \varphi < 2\pi.$$

a , b , c - полуоси по x , y , z соответственно (см рис. выше)

φ , θ - параметры

Параметры φ и θ являются углами поворота оси (в сферической системе координат) в горизонтальной и вертикальной плоскостях, поэтому для них заданы такие диапазоны.

Практическая реализация

Прим: язык реализации: C++.

Основой для выполнения лабораторной работы стала реализация куба с поворотом вокруг осей из предыдущей лабораторной работы. Поэтому функции модельно-видовых преобразований, переключение каркасного и твердотельного отображения, обработка нажатий клавиш не претерпели существенных изменений.

Главной особенностью реализации стало добавление классов вершин и граней и функции отрисовки эллипсоида.

Во-первых, для удобства были созданы классы Vertex и Face. Первый из них содержит два конструктора для создания вершин - напрямую через задание координат и через задание параметров для точки на поверхности конкретного эллипсоида.

```
1 class Vertex {
2     public:
3     float x, y, z;
4     float red, green, blue;
5     Vertex(float newx, float newy, float newz){
6         x = newx; y = newy; z = newz;
7     }
8     Vertex(float ell_v, float ell_u, float r, float g, float b){
9         x = 0.5*sinf(ell_v)*cosf(ell_u);
10        y = 0.6*sinf(ell_v)*sinf(ell_u);
11        z = 0.7*cosf(ell_v);
12        red = r;
13        green = g;
14        blue = b;
15    }
16 };
```

Объект класса Face хранит в себе все вершины, которые определяют грань, и имеет метод, который отрисовывает грань по вершинам в порядке их добавления.

```
1 class Face {
2     private:
3     std::vector<Vertex> vertexes;
4     public:
5     void addVertex(Vertex v){
6         vertexes.push_back(v);
```

```

7     }
8     void draw_face() {
9         glBegin(GL_POLYGON);
10        float *coord = new float[3];
11        for (int i = 0; i < vertexes.size(); ++i){
12            glColor3f(vertexes[i].red, vertexes[i].green, vertexes[i].blue);
13            glVertex3f(vertexes[i].x, vertexes[i].y, vertexes[i].z);
14        }
15        glEnd();
16        delete [] coord;
17    }
18 };

```

При этом цвет грани задается в совокупности цветов вершин, входящих в нее.

Параметрическое уравнение эллипсоида будет учитываться при создании точки, и, соответственно, главной задачей далее является генерация вершин и объединение их в грани в нужном порядке: грань определяется четырьмя соседними вершинами, положение которых отличается не более чем на один шаг по обоим параметрам.

```

1 void draw_ellipsoid() {
2     float step = M_PI/50;
3     float r = 0.05, g = 0.05, b = 0.3;
4
5     for (float v = 0; v <= M_PI + step; v += step){
6         for (float u = 0; u <= 2 * M_PI + step; u += step){
7             Face f;
8             f.addVertex(Vertex(v, u, r, g, b));
9             f.addVertex(Vertex(v + step, u, r, g, b));
10            f.addVertex(Vertex(v + step, u + step, r, g, b));
11            f.addVertex(Vertex(v, u + step, r, g, b));
12            f.draw_face();
13
14            if (u < M_PI) {
15                r += step / 4; b -= step / 4;
16            }
17            else {r -= step / 4; b += step / 4;}
18            if (v < M_PI / 2) g += step / 4;
19            else g -= step / 4;
20        }
21    }
22 }

```

Результатом выполнения программы является окно с изображением эллипсоида, удовлетворяющим всем пунктам условия:

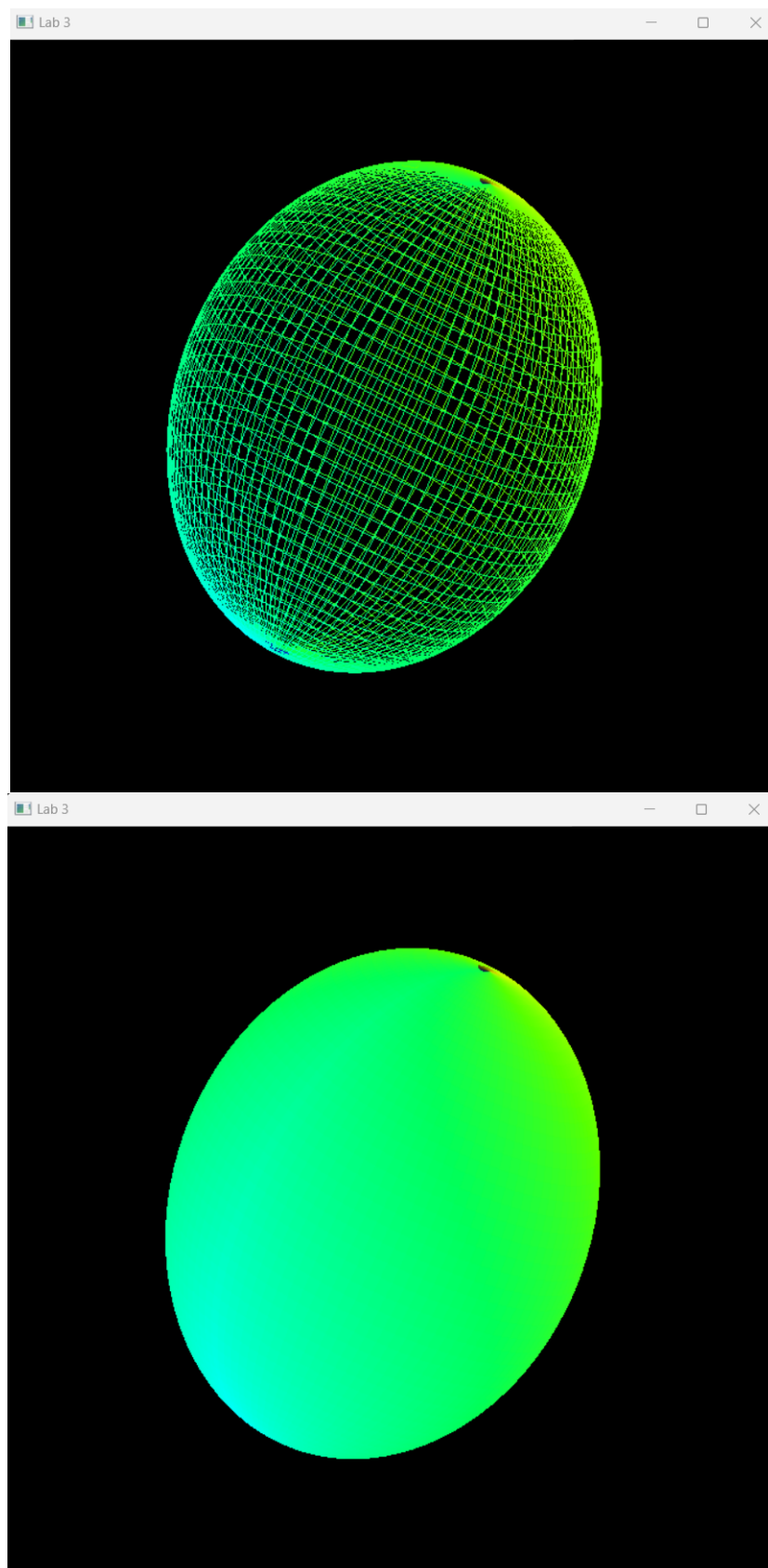


Рис.: Окно программы

Заключение

В ходе лабораторной работы были применены методы приближения поверхности, заданной параметрически, многоугольником. Результатом стало изображение эллипсоида, демонстрирующее, что при малом шаге параметров при разбиении фигуры получается реалистичное изображение, где ломаные и многогранники воспринимаются непрерывными гладкими кривыми и поверхностями.

Полный код реализации

язык C++

Файл primitives.h

```

1 #include <vector>
2 #include <GL/glut.h>
3 #include <iostream>
4
5 class Vertex {
6     public:
7     float x, y, z;
8     float red, green, blue;
9     Vertex(float newx, float newy, float newz){
10         x = newx; y = newy; z = newz;
11     }
12     Vertex(float ell_v, float ell_u, float r, float g, float b){
13         x = 0.5*sinf(ell_v)*cosf(ell_u);
14         y = 0.6*sinf(ell_v)*sinf(ell_u);
15         z = 0.7*cosf(ell_v);
16         red = r;
17         green = g;
18         blue = b;
19     }
20 };
21
22 class Face {
23     private:
24     std::vector<Vertex> vertexes;
25     public:
26     void addVertex(Vertex v){
27         vertexes.push_back(v);
28     }
29     void draw_face(){
30         glBegin(GL_POLYGON);
31         float *coord = new float[3];
32         for (int i = 0; i < vertexes.size(); ++i){
33             glColor3f(vertexes[i].red, vertexes[i].green, vertexes[i].blue);
34             glVertex3f(vertexes[i].x, vertexes[i].y, vertexes[i].z);
35         }
36         glEnd();
37         delete [] coord;
38     }
39
40 };

```


Файл main.cpp

```
1 #define _USE_MATH_DEFINES
2 #include <cmath>
3 #include "primitives.h"
4
5 static bool polygon_mode = true;
6 static GLfloat r_x = 0.0;
7 static GLfloat r_y = 0.0;
8 static int min_w_h;
9
10
11 void draw_ellipsoid(){
12     float step = M_PI/50;
13     float r = 0.05, g = 0.05, b = 0.3;
14
15     for (float v = 0; v <= M_PI + step; v += step){
16         for (float u = 0; u <= 2 * M_PI + step; u += step){
17             Face f;
18             f.addVertex(Vertex(v, u, r, g, b));
19             f.addVertex(Vertex(v + step, u, r, g, b));
20             f.addVertex(Vertex(v + step, u + step, r, g, b));
21             f.addVertex(Vertex(v, u + step, r, g, b));
22             f.draw_face();
23
24             if (u < M_PI) {
25                 r += step / 4; b -= step / 4;
26             }
27             else {r -= step / 4; b += step / 4;}
28             if (v < M_PI / 2) g += step / 4;
29             else g -= step / 4;
30         }
31     }
32 }
33
34 void rotate_ox(GLfloat phi){
35     GLfloat m_rotation_x[] = {1, 0, 0, 0,
36                               0, cosf(phi), sinf(phi), 0,
37                               0, -1*sinf(phi), cosf(phi), 0,
38                               0, 0, 0, 1};
39     glMultMatrixf(m_rotation_x);
40 }
41
42 void rotate_oy(GLfloat phi){
43     GLfloat m_rotation_y[] = {cosf(phi), 0, -1*sinf(phi), 0,
44                               0, 1, 0, 0,
45                               sinf(phi), 0, cosf(phi), 0,
```

```

46         0, 0, 0, 1};
47     glMultMatrixf(m_rotation_y);
48 }
49
50
51 void reshape(GLint w, GLint h) {
52     if (w > h) min_w_h = h;
53     else min_w_h = w;
54
55     glViewport(0, 0, min_w_h, min_w_h);
56     glMatrixMode(GL_PROJECTION);
57     glLoadIdentity();
58     rotate_oy(-M_PI/4);
59 }
60
61 void display() {
62     glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);
63
64     if (!polygon_mode) glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
65     else glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
66
67     glMatrixMode(GL_MODELVIEW);
68     glLoadIdentity();
69     rotate_ox(r_x);
70     rotate_oy(r_y);
71     draw_ellipsoid();
72
73     glFlush();
74     glutSwapBuffers();
75 }
76
77
78 void press(unsigned char button, int x, int y) {
79     if (button == 'a') r_y += 0.05;
80     else if (button == 'd') r_y -= 0.05;
81     else if (button == 'w') r_x += 0.05;
82     else if (button == 's') r_x -= 0.05;
83     else if (button == 'v') polygon_mode = (polygon_mode + 1) % 2;
84     glutPostRedisplay();
85 }
86
87
88 int main(int argc, char** argv) {
89     glutInit(&argc, argv);
90     glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
91     glutInitWindowPosition(80, 80);

```

```
92  glutInitWindowSize(500, 500);
93  glutCreateWindow("Lab 3");
94  glEnable(GL_DEPTH_TEST);
95  glutReshapeFunc(reshape);
96  glutDisplayFunc(display);
97  glutKeyboardFunc(press);
98  glutMainLoop();
99 }
```